OPENAI BUILD IT FORWARD HACKATHON

Hook: A Long-Form Technical Report and Case Study on Parallel AI-Assisted Development with Codex

Project narrative, engineering case study, and operational guide

Prepared from project source documents and repository artifacts
Hook development team
March 11, 2026

Abstract

This report presents Hook as both a software project and a case study in learning how to develop efficiently with OpenAI tools, especially Codex. Hook is a profile-first student support product that turns one student profile into prioritized next steps across academics, scholarships, research, internships, clubs, and campus support chat. The implementation story matters as much as the product story. The project used Codex not as a one-off code generator, but as part of a disciplined engineering workflow that included explicit repository setup, branch ownership, git worktrees, shared type contracts, parallel feature delivery, incremental prompting, diff review, compatibility fixes, merge control, synchronized documentation, layman translation, and exploratory QA planning. The report draws from the main collaboration setup notes, the OpenAI story document, the Engineer A catch-up plan, the Codex workflow draft, the Codex usage log, the newly provided Codex screenshots, and supporting repo artifacts such as the README, demo script, and judge preparation notes. It argues that the most valuable lesson from the project is operational rather than promotional: OpenAI tools were most effective when treated as collaborators inside a constrained system with clear context, narrow responsibilities, visible checkpoints, and human review. The report also examines the limits of that workflow, including time-estimation problems, cross-surface integration risk, and the distinction between product readiness and presentation readiness. The result is intended to serve two audiences at once. For judges, it provides a thorough technical narrative of how Hook was built with OpenAI support. For future engineering teams, it provides a practical case study in how AI can accelerate development without replacing technical judgment.

Contents

1. Executive Summary
2. Project Context and Constraints
3. Source Materials and Method
4. Hook as a Product System
5. Collaboration Architecture
6. Why Codex Was Introduced Into the Workflow
7. Phase I: Shared Scaffold and Contract Formation
8. Phase II: Parallel Delivery Across Worktrees
9. Phase III: Dashboard Integration and System Coherence
0. Phase IV: Productization Sprint as a Cross-Cutting Refactor
11. Creative Use of Skills Beyond Code Generation
12. Reliability Engineering and Demo Safety
13. What Codex Generated and What Humans Reviewed
14. Operational Patterns That Worked
15. Observed Failure Modes and Limits
16. A Practical Heuristic for AI-Assisted Delivery Time
17. Recommendations for Future Teams
18. Conclusion
19. Appendix A: Timeline of Major Codex Tasks
20. Appendix B: Prompt Anatomy and Why It Worked
21. Appendix C: Proposed Figure Program for Final Screenshot Insertion
22. Appendix D: References

Reader Guidance

This version is intentionally expansive. It is designed so a judge can skim the executive summary and contents, then jump directly to the sections on worktrees, merge control, productization, documentation, failure modes, or recommendations. It is also designed so that future screenshots can be inserted without changing the overall structure.

1 Executive Summary

Hook is a profile-first student support product. A student enters one profile once, and the system turns it into academic guidance, scholarship matches, research and internship opportunities, club recommendations, and support chat. The product itself is designed to feel coherent, but the stronger technical story lies in how it was built. Hook was developed under hackathon conditions using a three-worktree collaboration model and Codex-assisted implementation. That process allowed multiple contributors to make progress in parallel while still converging on one integrated application.

The central conclusion of this report is that Codex produced the highest leverage when it operated inside a disciplined engineering workflow rather than as a generic code generator. The project documents show several recurring patterns. Codex was effective when prompts included concrete repository context, explicit ownership, a stable type contract, clear acceptance criteria, and required documentation updates. It was particularly effective when each worktree represented a bounded responsibility set: platform and integration, academic and clubs, or opportunities. This structure made it possible to build multiple user-facing systems at once without serializing the entire repository through one branch.

The build also reveals important limits. AI-generated output was not the main bottleneck. Review, merge safety, contract migration, demo packaging, and fallback design were often more important than generation speed. The Codex workflow draft identifies time estimation as a weakness, and the Hook materials support that claim. Tasks framed as multi-hour often resolved much faster once repository state and definitions of done were explicit. More importantly, prompts framed too narrowly around deadlines can compress scope prematurely. Hook improved when the team switched to incremental prompts, explicit priorities, and frequent review checkpoints.

The report therefore treats Hook as both a product narrative and an engineering case study. For the product narrative, the key idea is that one student profile can coordinate several fragmented administrative domains. For the engineering case study, the key lesson is that OpenAI tools become more useful when surrounded by a strong operating model: shared contracts, parallel worktrees, human review, documentation discipline, prompt refinement, and explicit fallback strategies.

2 Project Context and Constraints

2.1 Hackathon context

Hook was built in a hackathon environment where speed mattered, but so did credibility. A common failure mode in hackathon software is to overbuild the appearance of intelligence while underbuilding the user flow, reliability story, or explanation of the system. The project materials show a different priority structure. The team aimed to build a practical, demoable student product and to show clearly how Codex accelerated the engineering process itself. This dual objective affected both technical decisions and documentation decisions.

2.2 Scope decisions

The system was deliberately scoped as an MVP. The ranking surfaces use deterministic local datasets rather than live connectors, and GPT-4o is used narrowly in chat rather than across the entire application. The materials repeatedly defend this choice. Local curated data made the ranking logic explainable, the user flow reliable, and the demo easier to control. The more ambitious productization branch introduced a Supabase-ready account and profile model, but even there the sprint documents explicitly instructed Codex to preserve demo fallbacks rather than overclaim unfinished production infrastructure.

2.3 Technical constraints

The major constraints included time pressure, multiple independent feature tracks, evolving profile contracts, and the need to turn repository state into presentation evidence. The team needed a workflow that could support platform work, academic logic, opportunities logic, and chat without forcing those tracks to wait on one another. At the same time, the final merged result needed to look like a single coherent product. This combination of constraints is why repository structure and process discipline matter so heavily in the report.

Constraint	Why It Was Significant	Project Response
Short delivery horizon	High-value work needed to land quickly without destabilizing the whole app.	Incremental prompts, branch ownership, and a merge owner on `main`.
Multiple major surfaces	The product spanned academics, scholarships, research, internships, clubs, dashboard, and chat.	Three worktrees with domain ownership boundaries.
Changing profile contract	The productization branch changed profile structure across many surfaces.	Dependency tracing, compatibility fixes, and a staged refactor sequence.
Presentation pressure	The team needed to explain Codex use credibly, not just ship code privately.	Documentation updates, logs, judge prep, and screenshot planning alongside implementation.

3 Source Materials and Method

This report is based primarily on four core materials: `docs/collaboration-setup.md`, `docs/openai-story.md`, the local copy of the attached Google document at `hook-engineer-a-catchup-plan.html`, and `docs/codex_workflow_report_draft.pdf`. These are supplemented by supporting repository artifacts that document the realized implementation and final demo preparation: `CODEX-LOG.md`, `README.md`, `docs/demo-script.md`, and `docs/judge-qa.md`.

The method used in writing this report is synthetic rather than chronological transcription. Where multiple documents described the same event or decision, the more technically specific and later-stage version was preferred. For example, the collaboration notes establish the ownership split and worktree architecture, the OpenAI story explains where Codex helped most, the catch-up plan shows how late-stage refactors were operationalized, and the workflow draft provides higher-level lessons about

prompt quality, merge coordination, and time estimation. The supporting repo artifacts then provide evidence that those plans were carried into the working repository and demo narrative.

This approach is consistent with the goal of the report. The purpose is not merely to summarize documents, but to extract the strongest engineering narrative that can be defended by them. That narrative is that Hook was built through a disciplined AI-assisted workflow, and that the workflow itself produced reusable lessons for future teams.

4 Hook as a Product System

4.1 Core product claim

Hook's core product claim is simple: student administrative work is fragmented, and one profile can organize that fragmentation into prioritized next steps. The application is designed around the idea that students should not need to restate their information on every surface. The profile is the organizing unit that powers degree planning, scholarship matching, research discovery, internship discovery, club matching, saved opportunities, and chat.

4.2 Why the product design matters to the engineering story

The product design matters because it directly shaped the repository architecture. A profile-driven system is only believable if the same profile truly carries across surfaces. This is why the shared type contract, profile context, and later removal of demo-profile leaks are so important. The more integrated the product claim became, the more important cross-surface consistency became as an engineering requirement.

4.3 Deterministic ranking plus narrow generation

The project materials repeatedly emphasize that GPT-4o was used narrowly. Chat is the only place where freeform generation is essential. All major ranking surfaces are deterministic and run over curated local datasets. This decision kept the system explainable and resilient. It also gave the team a more credible product story. Rather than claiming that all recommendations came from opaque AI inference, Hook could show transparent matching and reserve generation for tasks like routing, outreach drafting, and support responses.

[Insert screenshot/photo here: product overview, onboarding or dashboard hero, or one profile flowing into multiple surfaces]

Figure 1. Product overview frame showing Hook as a profile-first system. This image should make the one-profile-many-surfaces claim visually obvious and anchor the report before the workflow sections begin.

Surface	Technical Character	Role in the Product Story
Academic	Degree progress, prerequisite logic, and eligible-course analysis.	Shows that Hook is not only organizing information but prioritizing decisions.
Scholarships, research, internships, clubs	Deterministic profile-driven matching over local curated	Demonstrates that one profile can coordinate multiple domains of student

	datasets.	life.
Dashboard	Five-engine integration layer with fallbacks.	Acts as the single screen that proves the rest of the system is coherent.
Chat	GPT-4o plus local resource grounding and deterministic fallback.	Provides the flexible action layer without overextending generation into every surface.

5 Collaboration Architecture

5.1 Exact ownership split

The collaboration setup notes define a clear ownership model. Person A owned platform, onboarding, dashboard, chat, docs, and integration. Person B owned academic engine work, degree data, course metadata, and clubs. Person C owned scholarships, research, internships, opportunity data, and saved opportunities. This split appears consistently across the collaboration notes, the OpenAI story, and the repo artifacts.

5.2 Shared repository rules

The notes also define shared rules. Person A owns merges into `main`. Teammates should not edit another person's folders before merge windows. Shared type changes should update `lib/types.ts` first. Major Codex tasks should be logged in `CODEX-LOG.md`. Screenshots should be saved during the build. These are not ornamental rules. They are what turned the team into a coordinated system rather than a cluster of simultaneous edits.

5.3 Why worktrees mattered

The three-worktree setup is one of the strongest technical decisions in the project. It let contributors build against the shared type contract without waiting on one another's implementation details. Person A could continue platform work and later merge the finished surfaces into one demo-ready branch. The judge prep notes describe this plainly: three worktrees lowered conflict risk and allowed all five surfaces to merge cleanly into one profile-driven application.

[Insert screenshot/photo here: terminal or file explorer view showing three Hook worktrees and active branches]

Figure 2. Three-worktree development structure. This should be one of the centerpiece images in the final report because it directly supports the claim that the Codex workflow was parallel, branch-aware, and real.

6 Why Codex Was Introduced Into the Workflow

The source materials do not treat Codex as a generic productivity tool. They treat it as an engineering partner that can accelerate specific classes of work when given enough context. The OpenAI story identifies where Codex helped most: shared scaffold bootstrapping, profile state, onboarding, missing app-shell recovery, API route generation, matching logic, dashboard integration, chat, and final documentation. The catch-up plan adds another dimension by turning Codex into the primary executor of a late-stage productization sprint with explicit documentation obligations.

This is a more mature usage pattern than simply asking an AI assistant for snippets. Codex is effective in these materials because the team uses it inside a bounded, reviewed, and auditable workflow. The workflow draft captures this well: Codex performs best when execution is grounded in explicit repository state, stable team instructions, and fast human review. Hook supplies a concrete project-scale example of that principle.

The report therefore uses the phrase "Codex app workflow" deliberately. The application was not built by humans on one side and AI on the other. It was built by a combined system in which some tasks were planned and bounded by humans, many implementation drafts were produced by Codex, and critical product and merge decisions were validated by humans.

7 Phase I: Shared Scaffold and Contract Formation

7.1 Repo scaffold as technical infrastructure

The earliest tasks in `CODEX-LOG.md` show Codex helping initialize the collaboration scaffold, shared type contract, and teammate onboarding documents. This is technically significant because later parallelism depended on it. Separate worktrees only remain safe when they share one interface boundary. In Hook, that boundary was the profile contract and the agreed folder structure for app, components, APIs, data, docs, and shared libraries.

7.2 Why the shared contract mattered

The contract in `lib/types.ts` became the synchronization point for the rest of the repository. Platform code, academic matchers, opportunity matchers, dashboard integration, and chat all depended on it. When the projectization sprint later changed the contract, the resulting work touched many surfaces. That later refactor helps illustrate retroactively why a stable contract mattered from the beginning. It was not only a type file. It was the coordination mechanism that made parallel AI-assisted implementation viable.

7.3 Early documentation as part of the system

The same early tasks also created the documentation trail: collaboration notes, the Codex log, and later the README and OpenAI story. These artifacts are important because they show that the team understood from the beginning that process visibility was part of the deliverable. That choice later made it easier to convert the build process into a technical narrative without resorting to vague recollection.

[Insert screenshot/photo here: shared contract generation in lib/types.ts or early repo scaffold diff]

Figure 3. Shared type contract and scaffold creation. This image should show why branch isolation was possible: the branches did not diverge arbitrarily because they were all anchored to the same contract.

8 Phase II: Parallel Delivery Across Worktrees

8.1 Platform worktree

On the platform side, Codex generated the profile system, onboarding flow, dashboard shell, shared navigation, summary-card structures, and later the chat route and integrated dashboard. The OpenAI

story highlights an important debugging event during this phase: the app shell went missing, and Codex restored the Next.js 14 and Tailwind scaffold so the planned routes could run again. That incident matters because it shows that Codex was used for recovery and repair, not only generation.

8.2 Academic and clubs worktree

On the academic branch, Codex generated degree-plan and course metadata, the academic analysis engine, prerequisite alerts, and academic UI components. It also helped build the clubs dataset, deterministic club matcher, and clubs page. The resulting surfaces were not toy examples. The academic system was responsible for degree progress, prerequisite chains, eligible courses, and recommendation sections. The clubs system had to translate the profile into ranked recommendations with reasons. This shows Codex participating in nontrivial product logic and UI work at the same time.

8.3 Opportunities worktree

On the opportunities branch, Codex generated scholarship data and matching logic, then research and internship datasets, a shared opportunity matcher, and the user interfaces around them. Saved opportunities and deadline urgency features later extended this branch from simple listing toward a more persistent user workflow. The value of the worktree split is particularly clear here: opportunities work could evolve independently while academic and platform work continued in parallel.

Worktree	Primary Outputs	Why It Matters in the AI Case Study
`main`	Profile flow, onboarding, dashboard, chat, docs, merges.	Shows Codex operating at the integration layer rather than only on isolated features.
`feat/academic-clubs`	Degree data, academic analysis, clubs data, clubs matching, academic UI.	Shows bounded domain work proceeding independently on a parallel branch.
`feat/opportunities`	Scholarships, research, internships, opportunity matching, saved opportunities.	Shows AI-assisted development of data-heavy and user-facing features at the same time.

[Insert screenshot/photo here: side-by-side Codex sessions or browser tabs running in different worktrees]

Figure 4. Concurrent Codex execution across worktrees. This image is likely the strongest single piece of evidence for the report because it transforms the parallel-development claim from description into visible fact.

8.4 Why parallelism deserves emphasis

The workflow draft argues that Codex fits well into parallel repository workflows, and Hook demonstrates why. Worktrees turned Codex into a collaborator inside a branching strategy rather than a monolithic operator over the entire repository. That is a subtle but important point. AI does not automatically create parallelism. Parallelism came from the team's decision to give each branch a stable domain, a shared contract, and a merge owner.

9 Phase III: Dashboard Integration and System Coherence

9.1 The dashboard as proof of integration

Hook's dashboard is the strongest technical proof that the product became one coherent system. The judge preparation notes describe its behavior clearly: `app/dashboard/page.tsx` calls five APIs in parallel, then converts those responses into top-line summaries. This means the dashboard is not just a visual home page. It is the integration layer that proves separate worktrees actually converged.

9.2 Parallel API calls and unified profile context

The dashboard depends on the same shared student profile that also powers academic, scholarships, research, internships, clubs, saved opportunities, and chat. This is why the hardcoded-profile cleanup later mattered so much. A profile-first product becomes unconvincing immediately if some pages still ignore the real profile. The dashboard therefore represents both a technical and narrative checkpoint: if the dashboard works, the product appears unified; if it fails, the branch fragmentation becomes visible.

9.3 Why chat belongs on the dashboard

Chat is technically separate from the deterministic ranking surfaces, but placing it in the dashboard narrative is useful because it demonstrates the boundary between deterministic logic and narrow generation. The chat route uses GPT-4o plus local resource grounding and a deterministic fallback. That design preserves coherence with the rest of the system: the profile remains central, but generation is used only where it adds value.

[Insert screenshot/photo here: integrated dashboard with five summaries, chat panel, or fallback badge]

Figure 5. Integrated dashboard and unified student view. This image should make the final product architecture legible at a glance by showing multiple surfaces summarized from one profile.

10 Phase IV: Productization Sprint as a Cross-Cutting Refactor

10.1 Why the productization sprint matters

The Engineer A catch-up plan is one of the most valuable documents in the source set because it turns abstract AI workflow claims into a concrete refactor sequence. The document makes clear that Hook

already looked good as a demo, but still behaved like a hackathon prototype in visible ways: the profile was only stored in the browser, onboarding still asked demo-style questions, and some pages still showed a hardcoded fake student instead of the shared profile. The sprint goal was not to finish a full production plan. It was to make the product feel real enough to survive a live demo while also generating presentation-ready evidence of how Codex was used.

10.2 A staged, high-value sequence

The catch-up plan breaks the refactor into a staged set of prompts: shared contracts and auth foundation, onboarding rewrite, dashboard and navigation cleanup, removal of hardcoded demo-profile leaks, and a final integration and documentation pass. This sequencing matters because it mirrors the workflow draft's recommendation: prompts should look like engineering briefs, not vague rewrite requests. Each stage includes required files, non-negotiable behaviors, documentation obligations, and even presentation-friendly summary requirements.

10.3 Supabase-ready contract with controlled fallback

The OpenAI story documents the central change in this sprint: the MVP's session-only profile state was replaced by a Supabase-ready auth and profile foundation, while preserving a durable local fallback if credentials were not available. This was an important architectural compromise. It moved the product story closer to a real student account system while keeping the branch demo-safe. The catch-up plan explicitly instructed Codex not to overbuild. It asked for the thinnest practical implementation that still produced a convincing live demo by morning.

10.4 Identity cleanup as a trust issue

The hardcoded "Alex Rivera" leaks on academic and clubs pages are small in code size but large in narrative importance. The documents repeatedly describe them as demo-breaking or trust-breaking. Their removal is a useful case study in why cross-surface consistency matters more than feature count in a productized branch. The fix was not cosmetic. It required reconnecting those pages to the shared profile context and maintaining their loading and fallback behavior.

[Insert screenshot/photo here: productized onboarding or before-and-after identity cleanup evidence]

Figure 6. Productization sprint evidence. A good version of this figure would either show the updated onboarding flow or a diff or screenshot proving the removal of the old hardcoded profile behavior.

10.5 Product readiness versus presentation readiness

The Codex workflow draft introduces a useful distinction between product readiness and presentation readiness. Hook illustrates this clearly. By the end of the sprint, the core product changes needed for

the demo were largely in place: the account flow was more realistic, onboarding was more coherent, the dashboard was cleaner, and fake-profile leaks were removed. The remaining risk was concentrated in explanatory material, demo sequencing, and storytelling artifacts. This is an important lesson because it changes how teams should allocate their last hours before a demo. Once the product can run end to end, synchronized documentation can become the highest-value remaining work.

11 Creative Use of OpenAI and Codex Tools Beyond Code Generation

11.1 From prompts to operational skills

The strongest evidence for creative use of skills in Hook does not come from a named skill package inside the repository. It comes from the way the team converted repeated task classes into reusable Codex operating procedures. The catch-up plan is essentially a skill library written in prose. Each step defines a bounded task, files in scope, required documentation updates, what to preserve, and what to summarize at the end. The workflow draft describes this pattern at a higher level: reusable operational knowledge reduces prompt overhead and helps the agent stay reliable on recurring task families.

11.2 Documentation as part of the build system

The project repeatedly required documentation to update alongside code. The OpenAI story, demo script, judge preparation notes, README, and Codex log were all revised during implementation and again during the productization pass. This is a particularly strong example of non-code technical work. The documents were not purely narrative. They encoded demo safety rules, environment assumptions, product positioning, fallback explanations, and likely reviewer questions. In other words, the documentation acted as part of the system that made the demo coherent and believable.

11.3 Screenshot planning as a technical asset

The screenshot checklist in the OpenAI story is also an example of creative technical workflow use. Screenshots were not treated as decorative assets to be gathered at the end. They were planned as evidence of specific engineering milestones: shared types, profile state, dashboard shell, integration, academic data, academic engine, scholarships, clubs, opportunities, chat, and three-worktree execution. This transforms screenshot collection from presentation work into process instrumentation.

11.4 Daily Codex use as documented in the provided screenshots

The newly provided screenshots deepen this part of the report because they show Codex being used in several small but revealing ways that do not always appear in a repository diff. One screenshot shows Codex running a multi-step task list in the background while the user continued writing and planning. Another shows the user asking what else should be added to a message and whether a particular planning document should be referenced. Another asks Codex to explain partially completed steps in plain language for a nontechnical boss who mainly cares about demo risk. Another asks whether the agent can connect to a Chrome tab pointed at localhost and write a quick bug report after stepping through functionality. Together, these images show that Codex was used not only for implementation, but also for prompt refinement, explanation, risk translation, workflow design, and exploratory QA planning.

One screenshot is especially worth calling out because it captures the tone of the workflow. In the background-planning message, the user jokes that the message itself would "most likely be used as a prompt lol." That line should remain in the final narrative, not as a throwaway joke, but because it

captures a real phenomenon in AI-assisted development. Human coordination notes, if they are concrete enough, often become reusable machine instructions. The joke is funny because it is accurate. In Hook, status updates, planning notes, and report drafts were repeatedly promoted into Codex inputs.

[Insert screenshot/photo here: CODEX-LOG, screenshot checklist, or docs updated alongside code]

Figure 7. Documentation and evidence artifacts maintained during development. This figure strengthens the "Creative Use of Skills" story by showing that Codex was used for coordination, review, and presentation preparation, not only code generation.

[Insert provided screenshot here: Codex task UI showing "2 out of 5 tasks completed" while the report-writing task remains in progress]

Figure 8. Codex working through a structured task list while the report work is being coordinated. This screenshot is valuable because it shows Codex functioning as a background engineering agent with visible progress tracking rather than as a single-turn code assistant.

[Insert provided screenshot here: user asks whether to reference hook-ut-productization-plan.html and what else should be added]

Figure 9. Prompt refinement and source-selection during report preparation. This image supports the claim that Codex was used for iterative framing and message improvement, not only for code generation.

[Insert provided screenshot here: planning message that ends with "This message will also most likely be used as a prompt lol"]

Figure 10. Human coordination text becoming prompt material. This is the best place to preserve the team's joke. It shows that planning messages were intentionally close enough to executable instructions that they could later be fed back into Codex.

[Insert provided screenshot here: Codex task UI showing step-by-step productization work and "Review changes"]

Figure 11. Staged Codex execution for the productization branch. This screenshot strengthens the case that large refactors were decomposed into explicit, reviewable steps rather than handled as opaque rewrites.

[Insert provided screenshot here: user asks for a layman's explanation of partial completion for a boss concerned about the demo]

Figure 12. Codex used to translate engineering state for a nontechnical stakeholder. This matters because it shows OpenAI tools contributing to communication and risk management, not just implementation output.

[Insert provided screenshot here: user switches tasks and asks Codex to generate a new conversation prompt from the documentation set and rubric]

Figure 13. Codex used to generate a better Codex prompt. This screenshot is important for the case study because it shows a meta-workflow: the team used Codex to improve the quality of later Codex interactions.

[Insert provided screenshot here: user asks whether the agent can connect to a Chrome tab on localhost and produce a bug report]

Figure 14. Exploratory browser-driven QA planning with Codex. Even as a capability question, this screenshot is valuable because it demonstrates that the team was trying to extend Codex from coding into interactive validation and demo hardening.

Artifact	What It Did Technically	Why It Counts as Creative Workflow Use
`CODEX-LOG.md`	Tracked major Codex tasks chronologically.	Made the AI-assisted workflow reviewable and auditable.
`docs/openai-story.md`	Captured leverage points, ownership boundaries, and fallback logic.	Connected implementation state to presentation narrative.
`docs/demo-script.md` and `docs/judge-qa.md`	Preserved the demo path, fallback narration, and likely judge questions.	Turned presentation sequencing into a repeatable technical deliverable.

12 Reliability Engineering and Demo Safety

12.1 Why reliability had to be designed in

The Hook documents show that reliability was not postponed until the end. The system was designed so that a demo could remain honest and functional even when some dependencies were unstable. This is especially visible in the dashboard and chat architectures, but it also appears in the productization

branch where the Supabase-ready account flow preserves local fallback behavior when environment variables are absent.

12.2 Dashboard fallback strategy

The dashboard uses static payloads shaped exactly like the live API responses. The judge notes explain that if an engine fails, the dashboard falls back to static data for that surface and shows a badge. If the entire environment becomes unstable, the project can force full static mode with `NEXT_PUBLIC_DEMO_STATIC=true`. This is important for two reasons. First, it protects the live demo. Second, it preserves honesty because the UI signals when fallback behavior is active.

12.3 Chat fallback strategy

Chat follows the same pattern. GPT-4o is grounded with the profile and local campus resources, but if the model call fails or the key is unavailable, the route returns deterministic local guidance instead of a broken experience. The product therefore remains usable even when the only generative surface is degraded.

12.4 Thin backend, explicit narrative

The demo script makes an important rhetorical move: it does not hide thin backend areas such as browser-side parsing limitations. Instead, it explains them as reviewable fallback behavior. This should be recognized as a sound engineering choice. A resilient MVP is more credible than a brittle system that claims to be fully automated.

Figure 8. Demo fallback behavior in the integrated product. This figure helps show that reliability in Hook was engineered intentionally rather than improvised after integration.

13 What Codex Generated and What Humans Reviewed

13.1 Generated implementation draft

The OpenAI story is explicit about the split between Codex and human responsibilities. Codex generated major implementation drafts across the shared contract, profile state, onboarding UI, shared navigation, summary-card system, dashboard integration, demo fallbacks, academic and opportunity API support, chat, and final documentation. This is a broad scope. It shows that Codex was used across repo setup, product UI, data shaping, route generation, integration, and documentation.

13.2 Human review and decision-making

Humans remained responsible for scope cuts, dataset realism, merge order, UI and copy edits, fallback believability, and final pitch wording. This is crucial to state plainly. The project's own documents repeatedly insist that Codex was not a black-box code dump. Human contributors still judged product realism and merge safety. That is part of why the report can present the workflow credibly.

Codex-Weighted Work	Human-Weighted Work
Shared scaffold and type contract	Ownership split and scope decisions
Profile state and onboarding drafts	Data realism and curation quality checks
API routes and matching logic	Merge order and final integration decisions
Dashboard integration and chat implementation	UI and wording refinement for the live demo
Documentation rewrites and repo cleanup	Final judgment on whether fallbacks stayed honest

This division of labor is one of the reasons Hook works as a case study. It avoids the least convincing AI narrative, namely that the model "built everything." Instead, it shows a more realistic relationship between automation and supervision.

14 Operational Patterns That Worked

14.1 Explicit context management

The workflow draft identifies explicit context management as a major force multiplier. Hook supports that claim. The best prompts in the source materials included repository path, branch purpose, current gaps, definitions of done, files in scope, and documentation requirements. In the productization sprint, each prompt even specified what presentation-friendly summaries to produce afterward. This is much closer to a mini-runbook than a normal chat prompt.

14.2 Bounded branch ownership

Worktree structure alone was not enough. It only worked because ownership was bounded and shared files were controlled. The collaboration setup notes explicitly reserve merges and shared-contract edits

to Person A. This is consistent with the workflow draft's observation that branch-aware tasking and clearly partitioned ownership keep multiple development tracks moving without forcing one agent to touch the whole repo serially.

14.3 Diff-based review loops

The workflow draft also emphasizes diff-based review and inline correction rather than full reprompting. Hook's documents reflect that mentality. The catch-up plan instructs the operator to run prompts one at a time, review the diff, commit, and then move on. This approach is slow enough to preserve control but fast enough to keep momentum.

14.4 Docs alongside code

One of the strongest workflow patterns in Hook is that implementation and documentation moved together. The OpenAI story explicitly names this as part of the under-deadline strategy. Every major step updated not only code but also the repo narrative, demo notes, judge preparation, and Codex log. This ensured that the final explanation of the project was not fabricated at the end.

14.5 Fallback-conscious delivery

The team consistently preferred explicit fallback paths over fragile claims of completeness. This appears in dashboard static mode, chat fallback logic, local profile durability when Supabase is absent, and seeded upload parsing behavior. The result is a product that remains usable and explainable even under stress. This is an important lesson for AI-assisted development under deadline: trust is often improved by clearly defined fallback behavior.

15 Observed Failure Modes and Limits

15.1 Time estimation

The most direct limitation identified in the workflow draft is time estimation. The notes state that Codex often projected multi-hour work for tasks that in practice completed in roughly thirty to forty-five minutes. Hook's productization materials support the relevance of this issue because several late-stage fixes were scoped as large risk items but executed quickly once prompts became concrete and bounded.

15.2 Scope compression under deadline framing

The workflow draft makes a stronger point than simple estimate inaccuracy: prompts framed too strongly around limited human time windows can encourage the model to reduce scope prematurely. The practical implication is that teams should separate three things in prompts: the deadline, the minimum acceptable scope, and the behaviors that cannot be compromised. The better prompts in Hook do exactly this. They preserve fallback behavior, documentation synchronization, and end-to-end buildability while still admitting that the implementation can be thin.

15.3 Cross-cutting integration risk

AI-assisted development is easiest on isolated tasks and hardest on contract migrations. Hook's later productization pass is a good example. Once the profile contract changed, many surfaces needed adjustment at once. The OpenAI story explicitly calls out the need to map every `StudentProfile` dependency first and then apply compatibility fixes. This is a reminder that the main risk in AI-heavy workflows may lie not in code generation quality but in broad integration blast radius.

15.4 Documentation lag

The product readiness versus presentation readiness section of the workflow draft identifies another risk: the software can be more complete than the explanation of the software. Hook's own materials acknowledge this. Some support materials lagged until the final documentation pass. The team corrected this by requiring docs to update alongside major implementation steps, but the risk remained real enough to warrant a dedicated section in this report.

16 A Practical Heuristic for AI-Assisted Delivery Time

The Codex workflow draft proposes a useful planning heuristic: instead of asking how long the model will take to code, teams should estimate how long the combined system will take to implement, review, validate, and accept. Hook supports this view. Generation time was often not the dominant factor. Review, merge safety, build checks, fallback verification, and demo-path validation usually dominated the effective cost of a change.

Planning heuristic

> Estimated delivery time = generation time + review time + environment overhead + acceptance testing

Task Class	Observed Character in Hook	Recommended Review Posture
Targeted edit	Dashboard copy cleanup, a local profile leak fix, or a doc rewrite.	Review the diff directly and verify the narrowest affected path.
Medium feature or refactor	Onboarding rewrite, API route addition, or UI component set.	Check interfaces, state flow, and user-visible regressions across touched files.
Cross-cutting integration	Profile-contract migration, worktree merge integration, or productization pass.	Budget time for compatibility work, merge resolution, smoke testing, and narrative cleanup.

For future teams, the practical lesson is to classify work by blast radius, validation depth, merge complexity, and narrative importance. Hook's productization sprint was not expensive because generation was slow. It was expensive because it touched many surfaces and had to remain demo-ready while those surfaces were changing.

17 Recommendations for Future Teams

1. Use AI inside a repository operating model, not as a detached code-writing step.
2. Establish a shared contract early and treat it as the main synchronization point for parallel work.
3. Use worktrees or equivalent branch isolation when multiple contributors and multiple AI sessions are active.
4. Assign a merge owner and reserve shared files to that owner whenever possible.
5. Write prompts like engineering briefs: current state, files in scope, constraints, acceptance criteria, and verification.
6. Separate deadline pressure from minimum acceptable scope so prompts do not collapse the solution unnecessarily.
7. Review diffs incrementally instead of waiting for one large, opaque batch of changes.
8. Update documentation alongside code so product readiness and presentation readiness do not drift apart.
9. Engineer fallback behavior explicitly and make it visible rather than hiding it.
10. Treat screenshot collection and logs as technical evidence, not as decoration for the final slides.

These recommendations are intentionally pragmatic. They do not assume an ideal environment or a perfect model. They assume a team is moving quickly, trying to preserve quality, and needs AI to function inside a real software-delivery process.

18 Conclusion

Hook is a useful project to study because it shows both sides of AI-assisted development. On one side, it produced a compelling product narrative: one student profile powering academics, funding, opportunities, community, and support chat. On the other side, it produced a credible engineering narrative: Codex helping initialize the repository, generate implementation drafts, recover missing infrastructure, support parallel feature work, participate in contract migration, and synchronize code with documentation and demo preparation.

The strongest claim this report can make is not that AI made the team magically faster. It is that AI became more useful once the team supplied structure around it. Clear ownership, worktree separation, explicit prompts, shared contracts, review loops, and fallback design turned Codex into a practical engineering collaborator. That combination of leverage and discipline is what gives Hook both its product coherence and its instructional value as a case study.

Appendix A: Timeline of Major Codex Tasks

This appendix condenses the major entries in `CODEX-LOG.md` into a report-friendly table. The goal is not to reproduce every file touch exhaustively, but to show the scale, sequencing, and breadth of Codex-assisted work.

Checkpoint	Primary Focus	Representative Outcome
T+0:00 to T+0:28	Scaffold and profile foundation	Shared repo structure, type contract, profile system, onboarding flow, and provider wiring.
T+0:45 to T+1:18	Dashboard shell and academic engine	Initial dashboard shell, navigation, degree data, academic engine, and academic UI.
T+1:10 to T+2:05	Scholarships	Scholarship dataset, matcher, API route, and scholarship page UI.
T+1:46 to T+2:05	Clubs	Clubs dataset, deterministic matcher, API route, and clubs page.
T+2:50 to T+4:30	Research, internships, and saved opportunities	Opportunity datasets, shared matcher, UI surfaces, saved state, deadline urgency, and saved hub.
T+5:05 to T+5:40	Chat and final MVP integration	GPT-4o chat, local grounding, dashboard integration, static fallbacks, final docs, and screenshot checklist.
March 11 productization pass	Auth, onboarding rewrite, dashboard polish, leak cleanup, final packaging	Supabase-ready profile foundation, real account inputs, profile consistency fixes, and final demo narrative alignment.

[Insert screenshot/photo here: CODEX-LOG entries or timeline evidence]

Figure 9. Codex task log as a timeline artifact. This figure would help readers see that the workflow was incremental, structured, and sustained across the full build rather than concentrated in one burst.

Appendix B: Prompt Anatomy and Why It Worked

The workflow draft argues that the strongest prompts resembled engineering briefs rather than casual requests. The Hook source materials support that view. Across the collaboration notes, catch-up plan, and workflow draft, the most effective prompt structure consistently included the following elements:

1. Repository and branch context, so the task is anchored to real repo state.
2. A bounded goal, such as auth foundation, onboarding rewrite, dashboard cleanup, or identity-leak removal.
3. Files and interfaces in scope, especially when a shared contract is involved.
4. Non-negotiable behaviors to preserve, such as fallbacks, parallel API loading, or existing page layout.
5. Documentation obligations, so the repo narrative stays synchronized with the code.
6. A concise summary requirement, often including what remains and what to demo next.

```
Representative prompt anatomy

Context:
- current repo state
- branch purpose
- time window
- product goal

Goal:
- one bounded deliverable

Requirements:
- file-level or behavior-level constraints
- explicit fallback or reliability expectations

Documentation requirements:
- update log
- update OpenAI story
- update README, demo notes, or judge prep

Output:
- code changes
- docs changes
- concise operator summary
```

This structure worked because it reduced ambiguity and preserved continuity between code and narrative. The prompt became a local runbook for the task. In complex repo work, that is more useful than a generic request for help.

Appendix C: Proposed Figure Program for Final Screenshot Insertion

The final version of this report should include a deliberate figure program rather than scattered screenshots. The images should support the report's argument, not merely document the UI. The figure set below now reflects both project-state screenshots and the specific Codex-tool screenshots provided in the current conversation.

Figure	Recommended Content	Primary Purpose
Figure 1	Product overview or dashboard hero	Anchor the reader in Hook as a coherent product.
Figure 2	Three-worktree setup	Establish the parallel development model visually.
Figure 3	Shared type contract or scaffold generation	Show the technical foundation for safe parallel work.
Figure 4	Side-by-side Codex sessions	Provide the strongest evidence of simultaneous AI-assisted development.
Figure 5	Integrated dashboard	Prove that multiple branches converged into one user-facing system.
Figure 6	Productized onboarding or profile cleanup	Show the move from MVP shell toward a more credible product workflow.
Figure 7	Documentation and log artifacts	Support the "creative use of skills" narrative.
Figure 8	Fallback badge or static mode behavior	Show that reliability was engineered into the demo.
Figure 9	Prompt-refinement screenshot asking what else to add and whether to reference the productization plan	Show that Codex was used iteratively to improve its own instructions.
Figure 10	Planning message containing the joke that the message would probably become a prompt	Preserve the self-referential humor while also proving that coordination text fed directly into the AI workflow.
Figure 11	Codex task checklist with staged productization steps and review state	Show that cross-cutting work was decomposed into explicit steps rather than handled informally.
Figure 12	Layman's-explanation screenshot for a boss concerned about demo risk	Demonstrate use of Codex as a translation and stakeholder-communication tool.

Figure 13	Prompt-generation screenshot that references the report docs and rubric	Support the case-study point that Codex was used to improve later Codex sessions.
Figure 14	Chrome-tab and localhost QA question screenshot	Show the team's attempt to extend Codex into browser-level bug discovery and demo hardening.

Appendix D: References

1. `docs/collaboration-setup.md`
2. `docs/openai-story.md`
3. `hook-engineer-a-catchup-plan.html`
4. `docs/codex_workflow_report_draft.pdf`
5. `CODEX-LOG.md`
6. `README.md`
7. `docs/demo-script.md`
8. `docs/judge-qa.md`